Universidad ORT Uruguay
Facultad de Ingeniería

Entregado como requisito para la obtención del título de
Ingeniería en Sistemas

Juan Pedro Ferreira (292400)
Matías Trujillo (272561)
Agustín Lanata (285326)

Entrega 1
Diseño de Aplicaciones 2

2025

Índice

Evidencia de Clean Code y de la aplicación de TDD **3**
Estrategia de TDD seguida 3
Clean Code 3
Atributos (*) 4
CreateAttribute 4
Update_Attribute 4
Delete_Attribute 7
Lista de métodos que invoca (*) 8
Ejecutor de métodos: 10
Coverage Final: 11
Anexo: 12

Evidencia de Clean Code y de la aplicación de TDD

Estrategia de TDD seguida

Para desarrollar este proyecto consideramos utilizar la estrategia **inside-out**, esta comienza con la implementación de las partes más internas del sistema, como las entidades del dominio y sus repositorios. Luego, se avanza hacia el exterior, integrando estas partes en componentes más grandes y complejos.

Decidimos empezar con los conceptos que teníamos más claros, como WebApi era un tema nuevo para la mayoría, nos pareció mejor comenzar por el dominio. Para luego de adquirir los nuevos conocimientos, poder completar el proyecto.

A medida del proyecto consideramos que algunas entidades necesitaban un repositorio entonces fue agregado más avanzado en el proyecto, como el caso de “InvokedMethod”

Por ultimo para mantener una clara separación de los proyectos de prueba de los de la solución dentro de nuestro proyecto tenemos src/ y test/ para además de cumplir con esto genero una facilidad visual a la hora de desarrollar. También cabe recalcar que las clases de dominio se prueban mediante las pruebas de clases de logica de negocio es por esto que no contamos con Simulator.Domain.Test

Clean Code

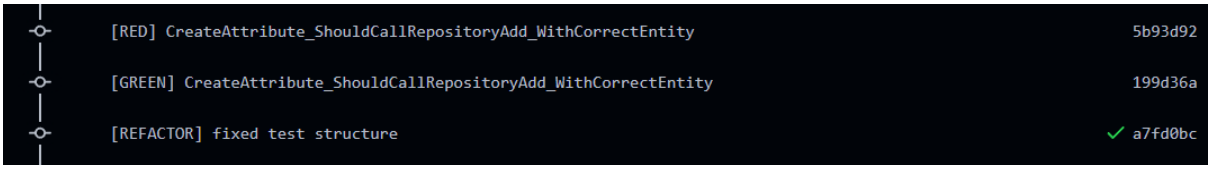
Durante el desarrollo de nuestro proyecto, seguimos varios principios de Clean Code para asegurarnos de que el código fuera fácil de leer, entender y mantener. Algunas de las prácticas que implementamos incluyen:

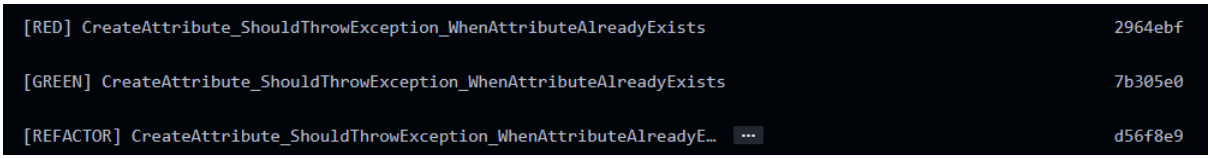
1. **Nombres claros y significativos**:
    - Asignamos nombres descriptivos a nuestras clases, métodos y variables, evitando abreviaciones o nombres confusos. Esto ayuda a que cualquier desarrollador que trabaje en el proyecto pueda entender rápidamente la funcionalidad de cada componente..
2. **Métodos pequeños y enfocados**:
    - Tratamos de mantener los métodos cortos y con una sola responsabilidad, para mejorar la legibilidad del código.
3. **Manejo adecuado de excepciones**:
    - Implementamos un manejo adecuado de excepciones, asegurándonos de capturar y gestionar los errores de manera eficiente sin saturar el

código con bloques try-catch innecesarios. Además, utilizamos tipos de excepciones específicas para diferenciar entre diferentes tipos de errores, manteniendo el código más claro y estructurado.

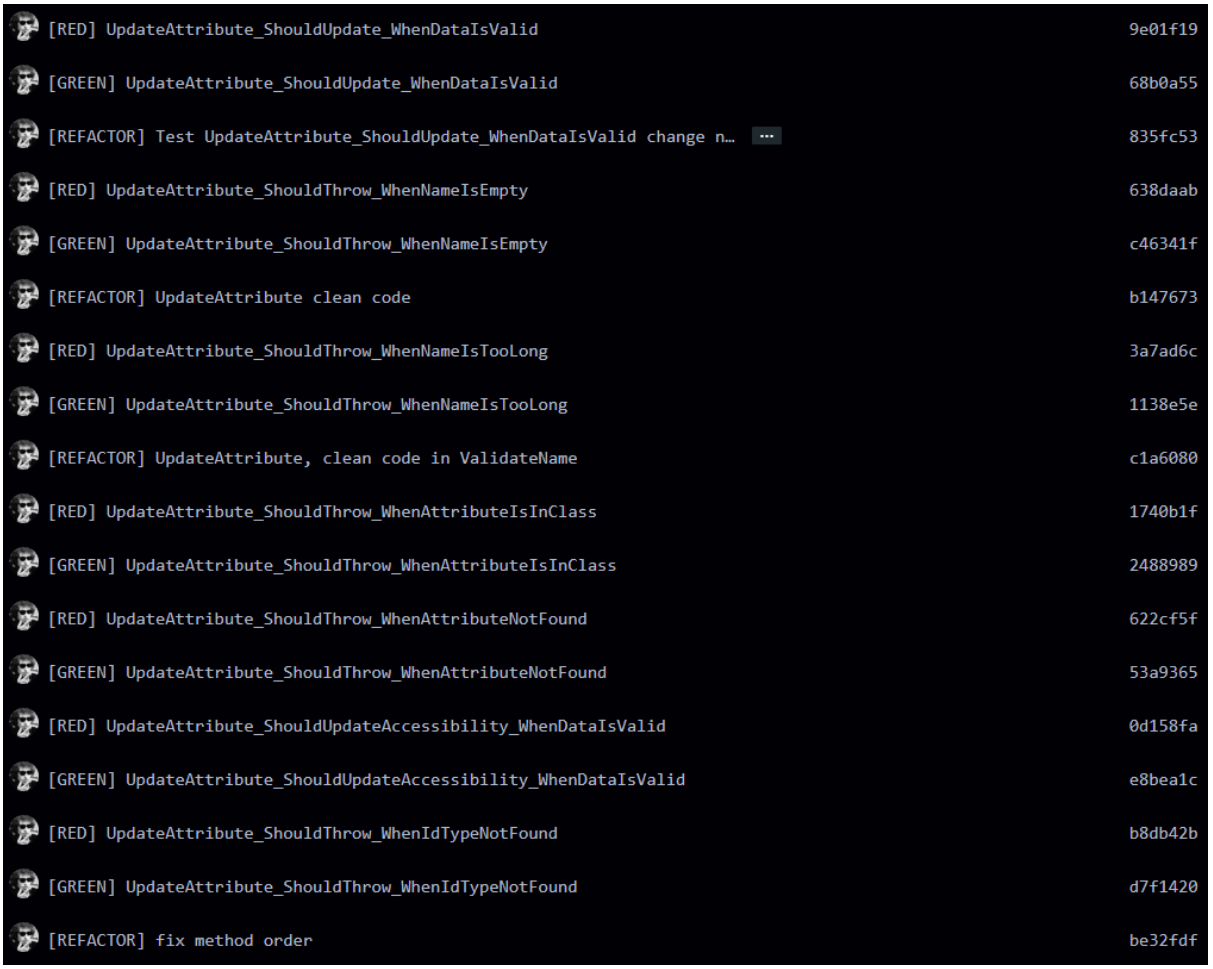
Atributos (*)

CreateAttribute



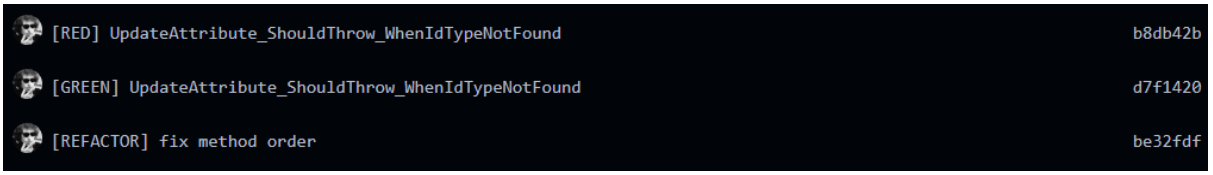


Update_Attribute



A modo de ejemplo

Ciclo seleccionado:



[RED]

Test

```
[TestMethod]
public void UpdateAttribute_ShouldThrow_WhenIdTypeNotFound()
{
    var id = Guid.NewGuid();
    var fakeTypeId = Guid.NewGuid();
    var attribute = new Attribute { Id = id, Name = "Attr" };
    var dto = new UpdateAttributeDTO { Name = "ValidName", Accessibility = "public", IdType = fakeTypeId };

    _mockAttributeRepository.Setup(r => r.Find(id)).Returns(attribute);
    _mockClassRepository.Setup(r => r.FindAll()).Returns([new Class { Id = Guid.NewGuid() }]); // different ID

    var ex = Assert.ThrowsException<MethodException>(() =>
        _attributeService.UpdateAttribute(id, dto));

    Assert.AreEqual("Return type not found.", ex.Message);
}
```

Funcion

```
public void UpdateAttribute(Guid id, UpdateAttributeDTO uAttribute)
{
    var attribute = _attributeRepository.Find(id);
    ValidateAttributeExist(attribute);
    ValidateAttributeIsNotInClass(attribute);
    ValidateName(uAttribute.Name, attribute);
    ValidateAccessibility(uAttribute.Accessibility, attribute);
    ValidateAttributeTypeUp(uAttribute.IdType);
    _attributeRepository.Update(attribute);
}
```

```
private void ValidateAttributeTypeUp(Guid? methodTReturnType)
{
}
```

[GREEN]

Sin cambiar el test se modifico la funcion

```
private void ValidateAttributeTypeUp(Guid? methodTReturnType)
{
    var classes = _classRepository.FindAll();
    if(methodTReturnType != null && classes.All(c => c.Id != methodTReturnType))
    {
        throw new MethodException("Return type not found.");
    }
}
```

En este ciclo el refactor se utilizó para reordenar las funciones. el cambio en este momento debería de haber sido de este estilo:

[GREEN]

```
public Class(string name, KeyWord? stereotype, Class? hierarchyClass, List<Attribute> attributes, List<Method> method)
{
    if (name == string.Empty)
    {
        throw new ClassException("The class name cannot be empty.");
    }

    Name = name;
    Stereotype = stereotype;
    HierarchyClass = hierarchyClass;
    Attributes = attributes;
    Methods = method;
}
```

[REFACTOR]

```
public Class(string name, KeyWord? stereotype, Class? hierarchyClass, List<Attribute> attributes, List<Method> method)
{
    if (name == string.Empty)
    {
        throw new ClassException("The class name cannot be empty.");
    }
    ValidatName(name);

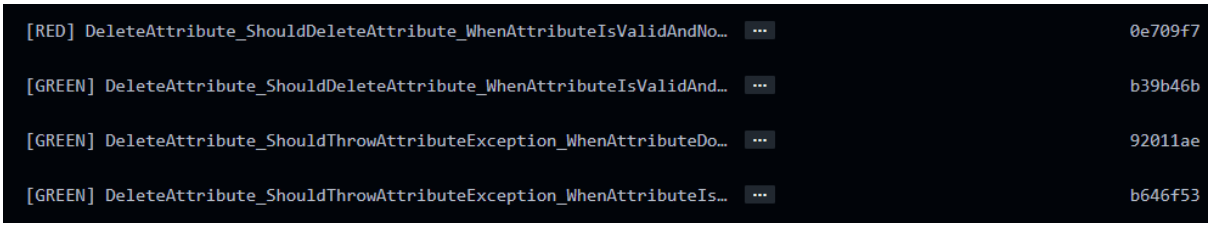
    Name = name;
    Stereotype = stereotype;
    HierarchyClass = hierarchyClass;
    Attributes = attributes;
    Methods = method;
}

private void ValidatName(string name)
{
    if (name == string.Empty)
    {
        throw new ClassException("The class name cannot be empty.");
    }
}
```

Coverage específico:

Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	95%	90%	✔
Simulator.DataAcces	100%	100%	✔
Simulator.Domain	99%	94%	✔
Simulator.DTO	97%	50%	✔
Simulator.WebApi	91%	89%	✔
Summary	**97%** (638 / 661)	**90%** (219 / 244)	✔

Delete_Attribute



Coverage específico tanto para DeleteAttribute como para CreatedAttribute

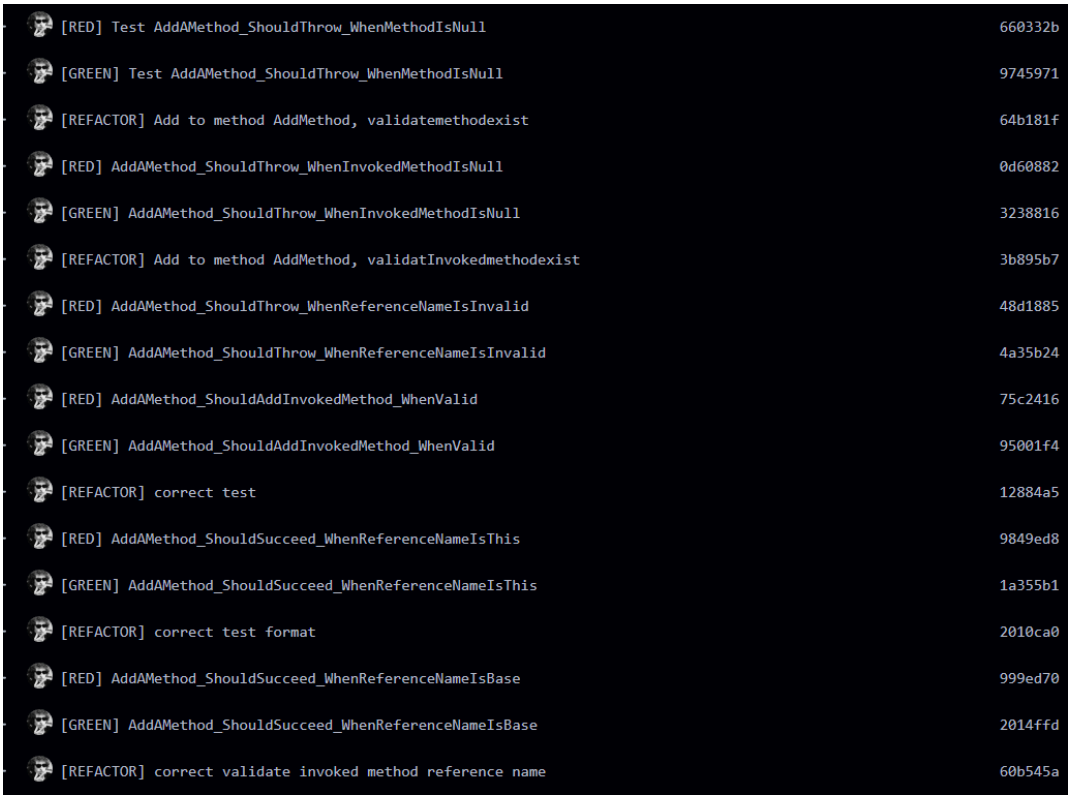
Code Coverage 97%

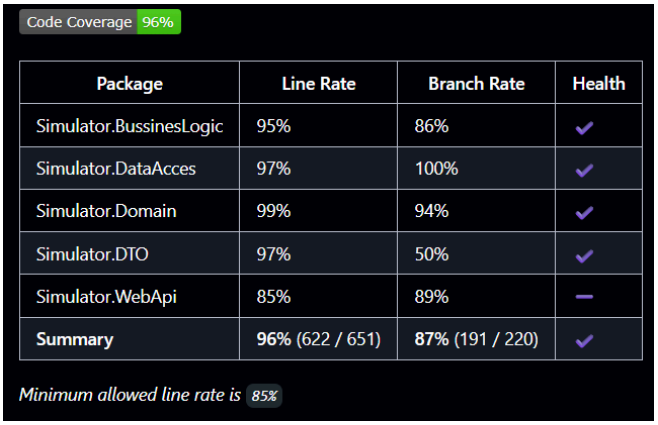
Package	Line Rate	Branch Rate	Health
DTO	71%	100%	✘
Simulator.BussinesLogic	100%	100%	✔
Simulator.DataAcces	98%	100%	✔
Simulator.Domain	100%	100%	✔
Simulator.WebApi	86%	100%	–
Summary	**97%** (203 / 210)	**100%** (12 / 12)	✔

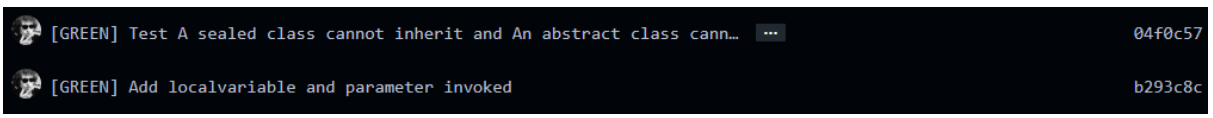
El bajo porcentaje en la columna "Line Rate" del paquete DTO (71%) se debe a que, por error, creamos la mayoría de los DTOs correspondientes con las entidades del

dominio, y como en ese momento se estaba probando únicamente el atributo, varias líneas de estos quedaron sin ser ejecutadas en las pruebas.

Lista de métodos que invoca (*)

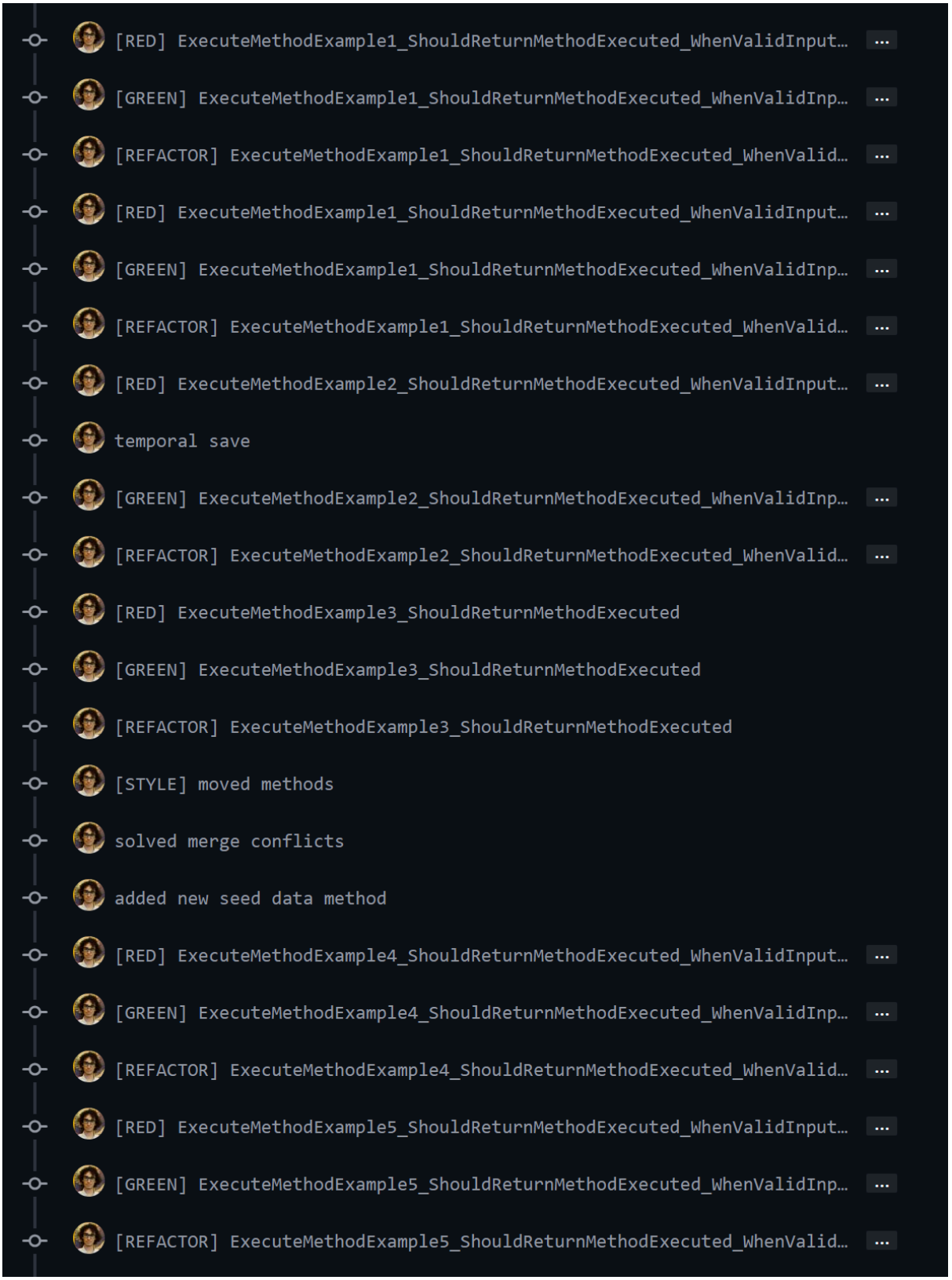






Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	97%	90%	✔
Simulator.DataAcces	100%	100%	✔
Simulator.Domain	99%	94%	✔
Simulator.DTO	97%	50%	✔
Simulator.WebApi	97%	93%	✔
Summary	**98%** (707 / 721)	**90%** (209 / 232)	✔

Ejecutor de métodos:

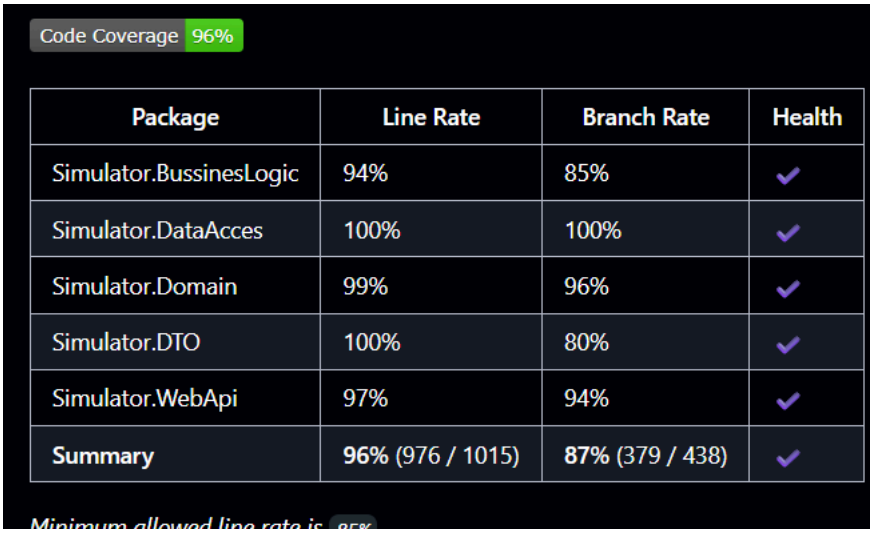


coverage especifico:

Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	96%	89%	✔
Simulator.DataAcces	100%	100%	✔
Simulator.Domain	99%	96%	✔
Simulator.DTO	100%	80%	✔
Simulator.WebApi	97%	94%	✔
Summary	**97%** (977 / 1004)	**90%** (384 / 428)	✔

Minimum allowed line rate is `85%`

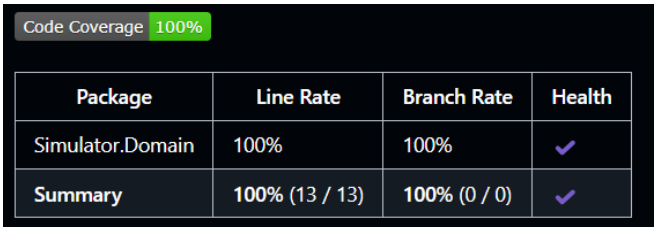
Coverage Final:



El coverage bajó un poco en la capa de Business Logic porque se agregó una funcionalidad al final del desarrollo y, por razones de tiempo, no se llegó a cubrir completamente con pruebas. La prioridad fue terminar de implementar esa parte, y está previsto corregir esto lo antes posible. A pesar de eso, como se puede ver en el anexo, el recorrido del proyecto muestra que el coverage se mantuvo en un nivel alto durante todo el desarrollo, lo que refleja el buen uso de TDD a lo largo del proceso.

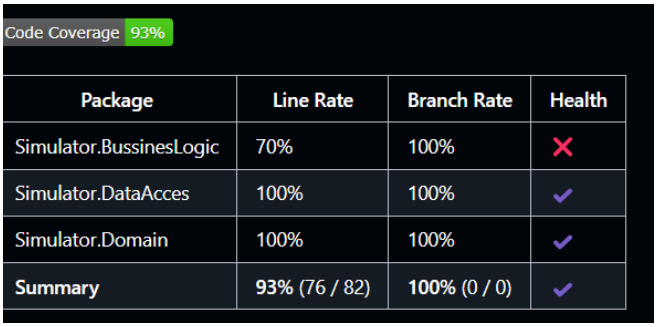
Anexo:

Evolución del TDD.



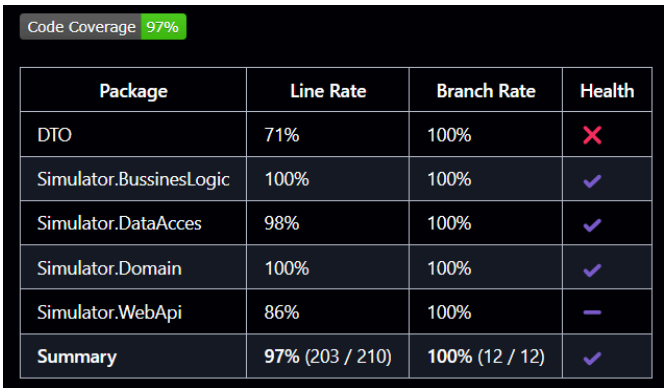
Code Coverage 100%

Package	Line Rate	Branch Rate	Health
Simulator.Domain	100%	100%	✔
Summary	**100%** (13 / 13)	**100%** (0 / 0)	✔



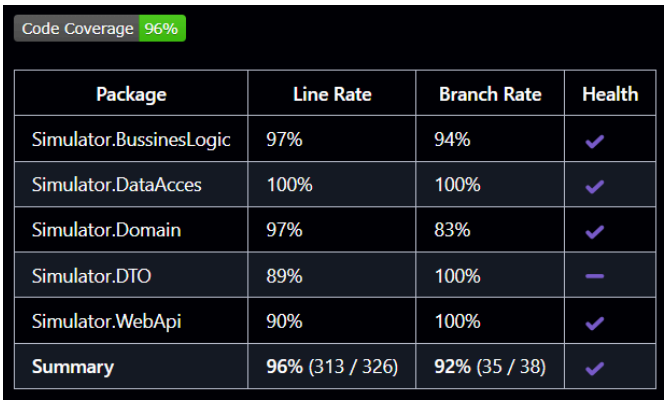
Code Coverage 93%

Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	70%	100%	✘
Simulator.DataAcces	100%	100%	✔
Simulator.Domain	100%	100%	✔
Summary	**93%** (76 / 82)	**100%** (0 / 0)	✔



Code Coverage 97%

Package	Line Rate	Branch Rate	Health
DTO	71%	100%	✘
Simulator.BussinesLogic	100%	100%	✔
Simulator.DataAcces	98%	100%	✔
Simulator.Domain	100%	100%	✔
Simulator.WebApi	86%	100%	–
Summary	**97%** (203 / 210)	**100%** (12 / 12)	✔



Code Coverage 96%

Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	97%	94%	✔
Simulator.DataAcces	100%	100%	✔
Simulator.Domain	97%	83%	✔
Simulator.DTO	89%	100%	–
Simulator.WebApi	90%	100%	✔
Summary	**96%** (313 / 326)	**92%** (35 / 38)	✔

Code Coverage 97%

Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	95%	90%	✔
Simulator.DataAcces	100%	100%	✔
Simulator.Domain	99%	94%	✔
Simulator.DTO	97%	50%	✔
Simulator.WebApi	91%	89%	✔
Summary	**97%** (638 / 661)	**90%** (219 / 244)	✔

Code Coverage 96%

Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	95%	86%	✔
Simulator.DataAcces	97%	100%	✔
Simulator.Domain	99%	94%	✔
Simulator.DTO	97%	50%	✔
Simulator.WebApi	85%	89%	–
Summary	**96%** (622 / 651)	**87%** (191 / 220)	✔

Code Coverage 98%

Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	97%	90%	✔
Simulator.DataAcces	100%	100%	✔
Simulator.Domain	99%	94%	✔
Simulator.DTO	97%	50%	✔
Simulator.WebApi	97%	93%	✔
Summary	**98%** (707 / 721)	**90%** (209 / 232)	✔

Code Coverage 98%

Package	Line Rate	Branch Rate	Health
Simulator.BussinesLogic	96%	89%	✔
Simulator.DataAcces	100%	100%	✔
Simulator.Domain	99%	96%	✔
Simulator.DTO	100%	80%	✔
Simulator.WebApi	97%	94%	✔
Summary	**98%** (824 / 843)	**90%** (283 / 314)	✔